

TP4, 5 et 6

Développement d'un jeu : Casse brique

1. Préambule

Lire ce document intégralement avant de commencer votre projet !

Le but de ce TP est de développer votre propre version du jeu classique Casse-brique (Breakout, 1976). Le développement se fera en Python avec le module Tkinter pour l'interface graphique. Vous avez 3 séances de 4 heures, travail en binôme, et programmation orientée objet.

La notation portera autant sur la qualité du code (découpage en classes, lisibilité, commentaires pertinents, utilisation de Git) que sur le résultat final.

Le but du jeu est de détruire tous les briques disposées en haut du canevas, à l'aide d'une balle rebondissante renvoyée par une raquette contrôlée par le joueur. La partie est perdue si la balle tombe en bas de l'écran.

Le rendu final de votre jeu pourrait ressembler à celui-ci :



2. Réalisation

2.1. Premiers pas

Dans un premier temps, il convient de mettre en place l'interface graphique principale qui devra comporter au minimum :

- Un canevas (zone principale du jeu)
- Une zone de texte affichant le score
- Un bouton permettant de démarrer une partie
- Un bouton permettant de quitter le jeu proprement
- Un menu avec différentes options

Avant de continuer, assurez-vous que le bouton «quitter» permet de sortir du programme proprement.

Vous pourrez ensuite :

- Insérer une raquette (rectangle ou image) en bas du canevas, contrôlable avec les touches gauche/droite du clavier.
- Ajouter une balle (cercle).

- - La balle se déplace automatiquement en diagonale.
- - Gérer les collisions avec les bords gauche/droite/haut (rebond).
- - Si la balle atteint le bas du canevas → vie perdue.
- Gérer la collision balle-raquette : la balle rebondit vers le haut en respectant la loi de Descartes.
- Ajouter une première rangée de briques (rectangles colorés) sur la partie haute du canevas.
 - Lorsqu'une balle touche une brique, la brique disparaît et la balle rebondit.
 - Le joueur marque des points.
- Ajouter plusieurs rangées de briques.
- Implémenter la notion de vies (par exemple 3 vies affichées quelque part dans la fenêtre).
- Fin de partie :
 - victoire si toutes les briques sont détruites
 - défaite si toutes les vies sont perdues.

2.1. Contraintes techniques

- Le jeu sera programmé en langage orienté objet
- Le jeu devra présenter une implémentation de liste, une de file et une de pile
- Le rendu se fera sous la forme d'une archive contenant l'ensemble de vos fichiers.
- Dans cette archive se trouvera un fichier *readme* indiquant les règles du jeu et les spécificités de votre implémentation, l'adresse de votre répertoire GIT, et où se trouvent les implémentations des structures de données demandées
- La notation prendra autant en compte le respect des consignes que le résultat final.

2.2. Pour aller plus loin au niveau fonctionnel :

Améliorations possibles, dans cet ordre ou dans un autre :

- Ajouter des niveaux impliquant des briques spéciales :
 - Brique à toucher 2 fois avant qu'elle ne se casse
 - Brique indestructible
 - Brique bonus/malus (donne une vie, agrandit la raquette, double la balle, réduit la taille de la raquette, réduit/accélère la vitesse de déplacement de la raquette, etc.)
- Introduire plusieurs niveaux de difficulté avec des dispositions différentes de briques.
- Augmenter la vitesse de la balle au fur et à mesure du jeu.
- Ajouter une image de fond différente pour chaque niveau.
- Sauvegarder les meilleurs scores dans un fichier texte.
- Permettre de modifier les touches de contrôle.
- Laisser libre cours à votre imagination et n'hésitez pas à demander conseil à vos aînés qui auraient perdu un temps précieux à jouer à ce jeu !